

Optimize Against EA

Handbook & User Manual

GameLab 2 — Julius-Maximilians-Universität Würzburg

Philipp Sehne

`Philipp.Sehne@stud-mail.uni-wuerzburg.de`

David Ahrens

`David.Ahrens@stud-mail.uni-wuerzburg.de`

July 13, 2026

Contents

1	Introduction	2
1.1	What is Optimize Against EA?	2
1.2	Who is this manual for?	2
1.3	Educational goals	2
2	Getting Started	4
2.1	Playing online (recommended)	4
2.2	System requirements	4
2.3	Running locally from source	4
2.4	First steps in the app	4
2.5	The built-in help and hint systems	5
3	Peak Finder	6
3.1	Concept	6
3.2	Game modes	6
3.2.1	Play	6
3.2.2	Create	7
3.2.3	Vs EA	7
3.3	The EA settings panel	7
3.4	The replay viewer	8
3.5	Sharing maps: map codes	8
3.6	Function Tuner	8
4	Maze Explorer	9
4.1	Concept	9
4.2	Playing the maze yourself	9
4.3	The experiment mode	9
4.4	Fitness functions: the headline experiment	10
4.5	Create mode	10
5	Shooter Game	11
5.1	Concept	11
5.2	Controls	11
5.3	Game modes	11
5.3.1	Solo	11
5.3.2	Horde	11
5.3.3	Community Raidboss	12
5.4	Reading the DNA display	12
5.5	How the evolution works (in brief)	12
6	A Small EA Glossary	13

7	Troubleshooting & FAQ	15
A	Route Reference	16

Chapter 1

Introduction

1.1 What is Optimize Against EA?

Optimize Against EA is an educational web application that teaches optimization concepts — in particular **Evolutionary Algorithms (EAs)** — through a collection of interactive mini-games. Instead of reading about selection, crossover and mutation in the abstract, players *experience* them: they compete against an EA, watch populations evolve step by step, and experiment with the algorithm’s parameters to see how behaviour changes.

The application contains three main mini-games, each highlighting a different facet of evolutionary optimization:

Peak Finder (/PeakFinder)

Probe a hidden function surface to find its global minimum — solo, or in competition with an EA whose every generation can be replayed and dissected step by step.

Maze Explorer (/MazeExplorer)

Explore an invisible maze by hand, then watch an EA solve the same maze — and discover how the choice of *fitness function* (Euclidean distance, geodesic distance, novelty search) completely changes the algorithm’s behaviour.

Shooter Game (/ShooterGame)

Fight waves of enemy agents whose behaviour is encoded as DNA and evolved by a genetic algorithm between rounds — the enemies literally adapt to *your* play style.

1.2 Who is this manual for?

This handbook is aimed at **players and instructors**: it explains how to start the application, how to play each game, and what each control and setting means. No programming knowledge is required.

Developers looking for the programmatic interfaces (engine functions, type contracts, worker protocols) should consult the separate **API documentation** instead.

1.3 Educational goals

Playing through the games, a player encounters the following concepts:

- search spaces, local vs. global optima, deceptive landscapes;
- the evolutionary loop: evaluation → selection → crossover → mutation → new generation;
- elitism and premature convergence;

- fitness shaping — how the *same* EA succeeds or fails depending on how fitness is defined;
- novelty search — why *not* chasing the goal can outperform chasing it;
- continuous genomes (points on a surface), discrete genomes (move sequences), and behavioural genomes (agent DNA).

Chapter 2

Getting Started

2.1 Playing online (recommended)

The easiest way to use the application is the live deployment:

<https://OptimizeAgainstEA.web.app>

No installation is required. The app runs entirely in the browser (client-side); a modern browser is the only prerequisite.

2.2 System requirements

- **Browser:** a current version of Chrome, Firefox, Edge or Safari.
- **Input:** mouse + keyboard *or* touch screen. No gamepad is required. The games are playable on desktop and mobile devices **[TODO: note current state of mobile support below 640px]**.
- **Network:** only needed to load the page (and for the community Raidboss feature, which uses Firebase).

2.3 Running locally from source

For local use (e.g. offline demos), the app can be built from source. Requirements: [Node.js](#) (developed with v24.x; v20+ should work) and `npm`.

1. Open a terminal in `OptimizeAgainstEA/OptimizeAgainstEAApp`.
2. Install dependencies: `npm install`
3. Start the development server: `npm run dev`
Vite prints a local URL (default <http://localhost:5173>); open it in your browser.

To create a production build instead, run `npm run build`; the static output in `dist/` can be served by any web server (`npm run preview` serves it locally).

2.4 First steps in the app

1. The **home page** (`/`) welcomes you and leads to the **Dashboard** (`/Dashboard`), the central game-selection screen.

2. Pick a game. Each game page has its own mode selector and settings.
3. Look for the **?** help buttons and the **Hints** toggle (light-bulb icon) — see Section 2.5.



Figure 2.1: The Dashboard — entry point to all mini-games.

2.5 The built-in help and hint systems

The application ships with two layers of in-app assistance:

Help dialogs (**?** buttons)

Open a modal with two tabs: **Gameplay** (how to play) and **Technical** (what the algorithm is doing). Available on the shooter modes and other game screens.

Contextual hints (light-bulb toggle)

When enabled, short coach-marks and toasts appear at the right moment during play — e.g. after your first probe in Peak Finder, a hint explains that the EA has just advanced a generation. Hints can be switched off at any time and re-armed with the **Reset** button. Your preference is remembered between visits.

Chapter 3

Peak Finder

3.1 Concept

Peak Finder (/PeakFinder) is a search-space exploration game in the spirit of *Battleships*. A function surface is hidden from you: every point on the map has a value between 0 (the global minimum — the point you are looking for) and 1 (far away from any minimum). The surface also contains *local* minima — traps that look promising but are not the true goal.

You explore by placing **probes**. Each probe reveals the surface in a small circle around it (as contour lines or a heat map), giving you partial information to plan your next probe. You win by placing a probe inside the **win radius** around the global minimum.

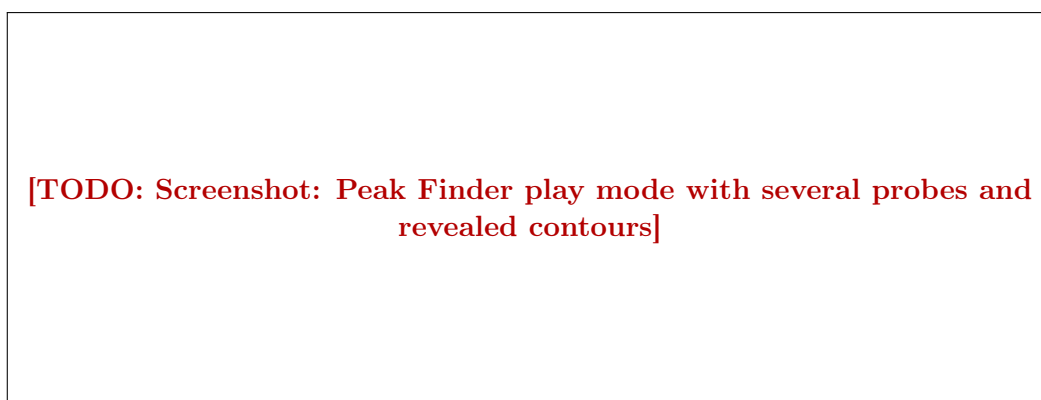


Figure 3.1: Peak Finder: probing a hidden surface. Contours are only revealed near your probes.

3.2 Game modes

Peak Finder has three modes, selectable at the top of the page:

3.2.1 Play

The classic solo mode. Load a map (a random one, a saved one, or one shared via code — see Section 3.5), then click/tap anywhere to probe. The interface shows your probe history and your best probe so far. Find the global minimum in as few probes as you can.

- **Probe:** click / tap a location on the map.
- **Visualization:** switch between **Contour** and **Heatmap** display of the revealed areas.
- **Win:** a probe inside the win radius triggers the win overlay, which shows your statistics.

3.2.2 Create

Build your own maps. Place up to 12 local minima on the map (a minimum spacing between minima is enforced), then choose which one is the *global* minimum. Dashed rings preview each minimum’s zone. Finished maps can be saved locally and exported as a **map code** to share with others.

3.2.3 Vs EA

The centerpiece mode: you and an Evolutionary Algorithm search the *same* hidden map — and this time it is a race.

- You probe as usual. For every probe you place, the EA advances a fixed number of **generations** (configurable).
- The EA maintains a **population** of candidate points. It wins when a sufficient fraction of its population (default 10%) gathers inside the win radius.
- After each of your probes you can open the **Replay** viewer to inspect exactly what the EA just did (Section 3.4).
- If the EA solves the map first, you may study its solution and try to beat it in a second attempt.

3.3 The EA settings panel

In Vs EA mode you control the algorithm you are playing against. The settings panel exposes the following parameters (defaults in parentheses):

Setting	Default	Meaning
Population size	40	Number of candidate points the EA maintains per generation. Larger populations explore more but converge more slowly.
Max. generations	200	Upper limit on generations before the EA gives up.
Crossover rate	0.8	Probability that two parents are recombined (instead of copied).
Mutation rate	0.3	Probability that an offspring is mutated.
Mutation strength	0.25	How far a mutation can move a point across the map.
Mutation decay	0.97	Per-generation factor shrinking the mutation strength — the EA takes big steps early and fine-tunes later.
Win fraction	0.10	Fraction of the population that must sit inside the win radius for the EA to win.
Selection	tournament	How parents are chosen: <i>tournament</i> , <i>roulette</i> or <i>elitist</i> .
Crossover	arithmetic	How two parents combine: <i>arithmetic</i> (blend), <i>uniform</i> or <i>single-point</i> .
Mutation	gaussian	Shape of the random perturbation: <i>gaussian</i> , <i>uniform</i> or <i>cauchy</i> (occasional long jumps).

Experimenting with these is the point of the game: try a huge mutation strength and watch the population scatter; switch selection to *elitist* and watch it converge prematurely into a local minimum.

3.4 The replay viewer

The replay viewer is a full-screen overlay that replays the EA’s last generations **phase by phase**:

initial → *sorted* → *elite* → *breeding* → *mutating* → *new generation* → *win check*

The left side shows the population as dots gliding across the map; the right side lists every individual with its fitness and a role badge (**ELITE**, **PARENT A**, **PARENT B**, **CHILD**, **WIN**). Use the playback controls to step forward/backward or let it autoplay. A fitness chart tracks the population’s best value across generations.

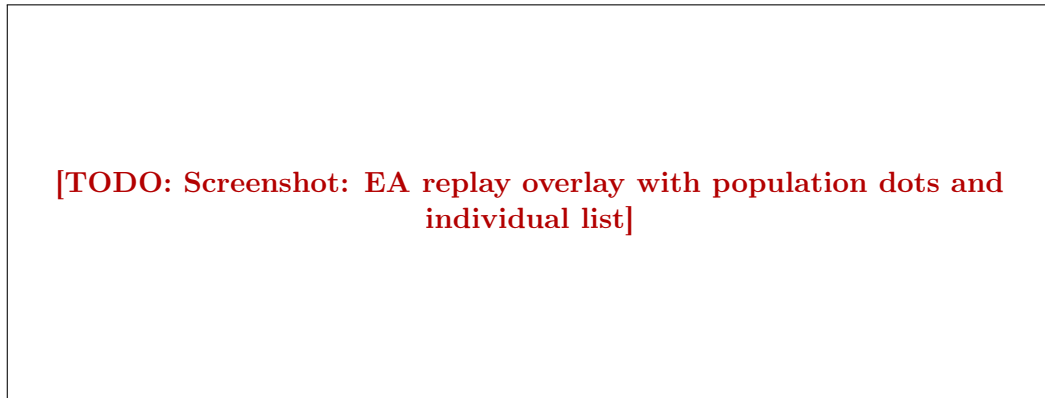


Figure 3.2: The replay viewer: every selection, crossover and mutation made visible.

3.5 Sharing maps: map codes

Every map can be exported as a short text code (via the **Code** dialog) and imported on any other device. Codes encode the minima positions, the global minimum and the win radius. Saved maps and saved function problems are also kept in a local sidebar for quick access.

3.6 Function Tuner

The `/FunctionTuner` page is a companion tool for adjusting the mathematical functions used as Peak Finder problems — useful for instructors preparing specific landscapes. **[TODO: Short description of the FunctionTuner workflow once its UI is final.]**

Chapter 4

Maze Explorer

4.1 Concept

Maze Explorer (/MazeExplorer) is a teaching exhibit built around a simple contrast: *you* explore an invisible maze step by step — then an EA explores the same maze with a whole population at once.

The maze is procedurally generated from a visible **seed**, so the exact same maze can be re-run under different settings — the key to the mode’s central experiment.

4.2 Playing the maze yourself

In **Play** mode the maze is hidden under fog-of-war. You move with the arrow keys (); each move reveals your current cell and its reachable neighbours. Reach the target cell to finish. Your explored trail is kept — it becomes the baseline against which the EA’s exploration is compared.

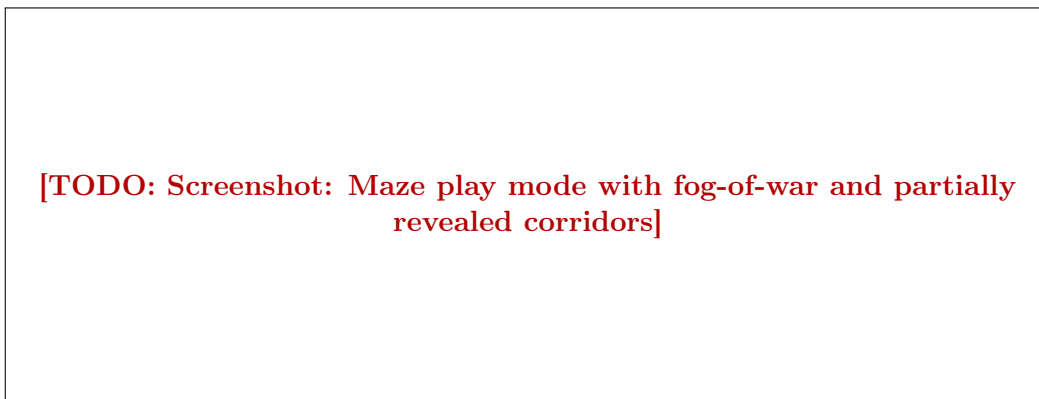


Figure 4.1: Exploring the maze by hand under fog-of-war.

4.3 The experiment mode

Experiment mode is the heart of the game. Here an EA solves the maze while you watch and dissect:

- Each **individual** in the population is a complete *path*: a fixed-length sequence of moves (). Moves into walls are simply ignored.
- The whole population is drawn as translucent **trails** on the maze: generation 1 is chaotic scribble; over generations the trails braid into a clean route to the goal.

- A per-generation **replay** (like Peak Finder’s, Section 3.4) dissects selection, crossover and mutation. Crossover is especially vivid here: the child follows parent A’s route up to the splice cell, then continues on parent B’s.

4.4 Fitness functions: the headline experiment

The maze offers several **selectable fitness functions**, and comparing them on the *same seeded maze* is the game’s main lesson:

Euclidean distance to goal

Rewards ending close to the goal as the crow flies. On most mazes this is *deceptive*: the population piles up against the wall nearest the goal and gets stuck. Teaches: deceptive landscapes and premature convergence.

Geodesic distance to goal

Rewards ending close to the goal *through the corridors* (true walking distance). The same EA now climbs cleanly to the exit. Teaches: fitness shaping can fix a broken search.

Novelty search

Ignores the goal entirely and rewards reaching cells that no previous individual has reached. Counter-intuitively, this often finds the exit when goal-chasing fails — the classic Lehman & Stanley maze result. Teaches: sometimes *not* aiming at the objective works better.

Length / efficiency penalty

Additionally rewards reaching the goal in fewer moves. Teaches: multi-objective tension between exploration and efficiency.

Suggested experiment: note the maze seed, run the EA with Euclidean fitness and watch it get trapped; then re-run the identical maze with geodesic fitness, then with novelty search. Same maze, same algorithm — three completely different behaviours.

4.5 Create mode

Create mode lets you generate and save mazes (seed-based) for later sessions. Saved mazes appear in a sidebar; like Peak Finder maps they can be shared via codes. **[TODO: Confirm final Create-mode feature set.]**

Chapter 5

Shooter Game

5.1 Concept

The Shooter Game (/ShooterGame) is a top-down arena shooter with a twist: the enemies are controlled by a **genetic algorithm**. Each enemy agent’s behaviour is encoded as a strand of **DNA** — seven genes controlling aggression, dodging, accuracy, preferred range, speed, aim prediction and fire rate. Between rounds the population *evolves*, using a recording (“ghost”) of your previous round as training data. The enemies do not just get harder — they adapt to *how you play*.

5.2 Controls

Action	Input
Move	W A S D (desktop) / virtual joystick (touch)
Aim	mouse (desktop) / aim zone (touch)
Shoot	mouse button (desktop) / aim zone (touch)

[TODO: Verify exact key/button bindings and any additional actions (dash, pause, ...).]

5.3 Game modes

The shooter lobby (/lobby/shooter) leads to three modes, each with its own ? help dialog in-game:

5.3.1 Solo

Round-based duels against evolved agents. After every round the population evolves against your ghost recording; the on-screen **DNA display** shows the current opponent’s genes, so you can literally watch traits like *aggression* or *accuracy* rise in response to your tactics.

5.3.2 Horde

Waves of enemies with map-based play (walls, pathfinding) on selectable maps. Enemy DNA evolves across waves. A companion **map editor** (/HordeMapEditor) lets you build custom horde maps. [TODO: Describe wave structure, mods/upgrades between waves, and win/lose conditions once final.]

5.3.3 Community Raidboss

A shared, community-evolved opponent: the boss’s DNA is trained by the rounds of *all* players (via Firebase), so everyone collectively shapes one adversary. **[TODO: Describe how a raidboss session starts and how player contributions are aggregated.]**

5.4 Reading the DNA display

Each agent’s genome is a vector of seven genes, all in $[0, 1]$:

Gene	Effect
Aggression	How strongly the agent chases you.
Dodge weight	How hard it tries to avoid your bullets.
Shoot accuracy	Aim precision (1 = perfect aim).
Preferred range	The combat distance it tries to maintain.
Movement speed	Speed bonus on top of the base speed.
Predict lead	How far it leads its shots ahead of your movement.
Fire rate	How quickly it shoots.

Watching these values drift over rounds is the game’s evolutionary storytelling: camp in a corner and *accuracy* and *range* genes win; rush constantly and *dodge weight* rises.

5.5 How the evolution works (in brief)

After each round every agent receives a fitness score based on its performance against you (damage dealt, survival, ...). The algorithm then applies tournament selection, single-point crossover and per-gene mutation to produce the next generation. Before your first round, the population can be pre-trained by simulating agents against each other, so even round 1 opponents are not purely random.

Chapter 6

A Small EA Glossary

Plain-language definitions of the terms used across all three games.

Individual

One candidate solution. A point on the map (Peak Finder), a move sequence (Maze), or a DNA vector (Shooter).

Population

The set of individuals alive in one generation.

Fitness

A number scoring how good an individual is. In Peak Finder lower is better (distance-like); in the Shooter higher is better.

Generation

One full cycle of evaluate $\rightarrow$ select $\rightarrow$ breed $\rightarrow$ mutate.

Selection

Choosing which individuals get to reproduce. *Tournament*: pick a few at random, the best of them wins. *Roulette*: chance proportional to fitness. *Elitist*: only the best reproduce.

Crossover

Combining two parents into a child. *Arithmetic*: blend their values. *Single-point*: take the first part from one parent, the rest from the other. *Uniform*: coin-flip per gene.

Mutation

Randomly perturbing a child. Keeps the population exploring instead of collapsing onto one solution.

Elitism

Copying the best few individuals unchanged into the next generation so progress is never lost.

Local optimum

A solution better than all its neighbours but worse than the global best — where greedy search gets stuck.

Deceptive landscape

A problem where following the apparent gradient leads *away* from the true optimum (see the Euclidean maze fitness, Section 4.4).

Premature convergence

The whole population collapsing onto one (often local) optimum before exploring enough.

Novelty search

Selecting for *new behaviour* instead of fitness — rewarding individuals that reach places no one has reached before.

Chapter 7

Troubleshooting & FAQ

The page loads but a game screen stays blank.

Make sure your browser is up to date (the app uses modern Web Worker and Canvas APIs).
Try a hard reload (`Ctrl+Shift+R` / `Cmd+Shift+R`).

The EA in Vs EA mode never seems to move.

The EA only advances when *you* probe — it runs a fixed number of generations per probe.
Place a probe, then open the replay viewer.

Hints stopped appearing.

One-time hints are only shown once per session. Use the **Reset** button next to the hint toggle to re-arm them, and check that the light-bulb toggle is on.

My saved maps / mazes disappeared.

Saved content is stored in your browser's local storage. It does not transfer between browsers or devices, and clearing site data deletes it. Use map codes (Section 3.5) to back up or transfer maps.

Performance is poor on my machine.

Reduce the EA population size and the visualization resolution settings. The EA itself runs in a background Web Worker, so the UI should stay responsive even with heavy settings.

Does the app collect data?

The app runs client-side. Only the community Raidboss mode communicates with a server (Firebase) to share evolved DNA. **[TODO: Confirm the exact data sent by the Raidboss feature.]**

Appendix A

Route Reference

Route	Page
/	Home / welcome page
/Dashboard	Game selection dashboard
/PeakFinder	Peak Finder (Chapter 3)
/MazeExplorer	Maze Explorer (Chapter 4)
/ShooterGame	Shooter Game (Chapter 5)
/lobby/shooter	Shooter lobby / mode selection
/HordeGame	Horde mode
/HordeMapEditor	Horde map editor
/FunctionTuner	Function tuning tool for Peak Finder problems
/Settings	Global settings
/Analytics	Analytics page